

AI Coding 关键跃迁

两年四次跃迁，程序员的身份被重写了四遍

在 AI 编程的无数概念中,真正**重塑了程序员身份**: 觉醒 → 狂热 → 约束 → 编排。

Ethan Jiang

四次跃迁总览

代理 workflow

Agentic Workflow

2024

编码者 → 管理者

氛围编码

Vibe Coding

2025.2

管理者 → 验收官

规范驱动开发

Spec-Driven Dev

2025 H2

验收官 → 架构师

代理工程

Agentic Engineering

2026.2

架构师 → 编排者

我们正处于第三次跃迁向第四次跃迁过渡的阶段。

- ❏ **贯穿四次跃迁的一条暗线：**模型能力每隔数月跃进一次,每一套"最佳实践"都自带保质期。没有放之四海而皆准的答案——只有结合你的团队认知和项目本质的持续适应。

跃迁 1 — 代理 workflow Agentic workflow (2024)

AI 从工具变为数字自主体

吴恩达(Andrew Ng)的关键论断:

"未来 AI 的进步将更多来自工作流的优化,而非参数的增长。"

GPT-3.5 配合代理 workflow,可以超越 GPT-4 的零样本直接输出——工程架构比模型本身更重要。

反思 Reflection

工具使用 Tool Use

规划 Planning

多代理协作 Multi-Agent

四大代理模式成为行业标准词汇。

跃迁 1 — 为什么这是一切的起点

从"被调用"到"被委托"

吴恩达之前

AI 是一问一答的聪明补全,无状态,无记忆。

吴恩达之后

AI 是能自主思考、使用工具、检查错误的 Agent。

没有"代理"这个认知转变,后续一切都不会发生——你没法对工具"放手"(就没有 Vibe Coding),不需要给工具"定规矩"(就没有 SDD),更不需要给工具"造生产线"(就没有代理工程)。

吴恩达的四种模式在当时是金科玉律——但仅仅一年后,模型自身已经学会了更复杂的规划和推理,框架正在被超越。**每一次跃迁的"答案"都有保质期。**

跃迁 2 — Vibe Coding (2025.2)

一条推文引爆的方法论内战

既然 AI 已经是能自主思考的 Agent，一个激进的问题浮出水面：我们能不能彻底放手，连代码都不看了？

*"我称之为 *vibe coding*,完全沉浸在氛围中.....忘记代码的存在。"*

— Andrej Karpathy (OpenAI 联合创始成员、前 Tesla AI 总监)

这不是一个普通人说的话。当 AI 领域的图腾级人物说"忘记代码",整个行业被迫选边站。

拥抱派的数据很诱人：44% 开发者手写代码不足 10%,Y Combinator 1/4 创业公司 95%+ 代码由 AI 生成。

然后,灾难来了。

跃迁 2 — 真实的灾难与冰冷数据

审查的负担超过了生成的速度



核心减速

资深开发者反而慢了 **19%**——尽管自认为更快了



代码崩塌

AI 共同编写代码的重大缺陷率激增 **1.7 倍**



安全风险

AI 生成代码约 **45%** 含安全漏洞



知识枯竭

Stack Overflow 月提问量暴跌 **90%**, 文档流量 -40%

代码产出暴增,但人类的审查带宽没有增长。更多代码被"接受"而非"理解"。

灾难的根因不是程序员不努力——是模型能力跑在了组织适应力前面。每个团队消化新能力的速度不同,这里没有统一节奏。

跃迁 2 — 文化冲击波与选边站

争论焦点已从"用不用"变成"怎么用"



拥抱派

Karpathy, Replit

"接受指数级变化,忘记代码的存在"



纪律派

Kent Beck, Simon Willison

"AI 是精灵,需要瓶子——测试与规范"



务实派

大多数一线开发者

60% 使用 AI,但仅 0-20% 完全委托

Kent Beck 发现了一个荒诞现象：AI 会删除测试来让测试通过。

到 2025 年底,连 Karpathy 自己都修正立场。没有人能退回"不用 AI"的时代——但也没有人能给出一个通用的"正确用法"。答案因团队而异。

跃迁 2 — 认知债务(Cognitive Debt)

比技术债务更危险：你不知道自己不知道什么

当你"Accept All"让 AI 写完代码而不去理解它,你积累的不是技术债务——而是**对自己系统的无知**。

❏ 隐性契约被打破：**程序员必须理解自己系统的运行逻辑**。当这个契约被打破,任何后续变更都变成在黑暗中摸索。

15 年积累的公开知识体系——多元解法、权衡分析、边界案例、社区审查——正在消散为私有的 AI 对话。

AI Coding本质上都在做同一件事：**在不手写代码的前提下,重建人类对系统的理解力和控制力**。

这种认知债务的痛感，催生了**第三次跃迁**——如果人类不再逐行写代码，那就必须找到新的方式来锚定意图、重建控制。答案是：**规范**。

跃迁 3 — TDD 的文艺复兴

从测试驱动开发到测试驱动生成 (TDG)

Kent Beck (TDD 之父) 提出 Augmented Coding (增强编码): 人类可以不亲自打字写实现代码, 但必须深度参与——尤其是定义边界和验证行为。

传统 TDD

人写测试 → 人写代码 → 测试验证代码

TDG (测试驱动生成)

人写测试 (边界) → AI 写代码 (内容) → 测试验证 AI

❏ 荒诞发现: AI 为了让测试通过, 会直接去修改甚至删除测试代码。这恰恰证明了测试必须由人类掌控——它是唯一不能被 AI 自行篡改的契约。

在 AI 时代, TDD 没有消亡, 反而成了抵御 AI 幻觉的唯一底线约束。人写边界, AI 填内容——这正是规范驱动开发的第一块基石。

跃迁 3 — 规范驱动开发 SDD (2025 H2)

代码不再是核心资产,规范才是

核心假设:

当生成代码的成本趋近于零,代码只是规范的编译产物(Compiled Artifact)。

Martin Fowler 的三级模型:

L1 规范先行 Spec-First

先写规范,再让 AI 生成代码

L2 规范锚定 Spec-Anchored

规范与代码同步维护,规范是"锚"

L3 规范即源码 Spec-as-Source

人类只编辑规范,永远不直接改代码

既然审查 AI 生成的代码太累,不如在源头把规矩定死。造砖免费的时代,最值钱的是蓝图。

跃迁 3 — 项目指令文件：SDD 的自发大规模采纳

千万开发者已经在做 SDD 而不自知

开发者自发在项目目录建 markdown 文件,告诉 AI "在这个项目里应该怎么工作"。各平台只是把这个草根行为收拢为正式功能——Anthropic 叫 CLAUDE.md,OpenAI 叫 AGENTS.md,Cursor 叫 Rules。名字不同,本质一样：**在 AI 写代码之前,先给它明确的约束和上下文。**这就是最轻量的 SDD。



10x

规划速度
提升



85%

重写频率
减少

当前主流层级：多数团队 L1,L2 开始普及

跃迁 3 — 生态大爆发：SDD 工具链的全面落地

巨头、创企与 Agentic IDE 的竞速

规范驱动开发（SDD）已不再是纸上谈兵。在 2025 年底至 2026 年初，一整套支撑 SDD 的生态工具链迅速成型，呈现出鲜明的分层特征。

Amazon Kiro

AWS 内置 SDD 工作流代理，将规范定义与云端流水线深度绑定

GitHub Spec Kit

通过官方生态确立“规范即源码”的三步流程与不可变原则

Tessl

Snyk 创始人下场创办，建立 10,000+ 开源库机器可读规范（Spec 注册表），致力于消灭 API 幻觉

Cursor / Windsurf

内建 Rules 与强下钻上下文管理的 Agentic IDE，一线开发者践行轻量级 SDD 的首选武器

Qodo Gen

“先生成 Spec 与测试，再生成代码”的高覆盖率测试倒推代码范式，异军突起

从云端基础设施到桌面 IDE，SDD 正在从理念变成每个开发者日常触碰的工具链。但最大的变量来自另一个方向——开源社区。

跃迁 3 — 开源社区的去中心化标准

OpenSpec: 连接人类意图与多模型底座的通用接口

工具无关 Tool-Agnostic

不受任何单一 IDE 绑架，支持搭配 Claude Code、Copilot 等二十余种 AI 使用

轻量指令协议

通过 /openspec:proposal 等轻量指令库，强制 AI 在动代码前与人类达成 Markdown 协议

通用接口

成为连接人类意图与多模型底座的最佳"通用接口"，是当前社区极具爆发力的流行标准

📌 **ThoughtWorks 技术雷达警告：**SDD 虽成行业共识，但目前各家工具生态割裂、规范文件的诠释差异依然很大。在标准一统江湖之前，采用像 OpenSpec 这样与模型解耦的开源标准，是抵御供应商锁定（Vendor Lock-In）的最优解。

当巨头各自为政时，开源社区正在用"与模型解耦"的方式，悄然建立 SDD 的事实标准——这可能是整个工具链生态中最值得押注的变量。

跃迁 3 — 过渡地带：我们正在这里

规范管意图,谁来管执行?



SDD 解决了

- AI 应该写什么代码(意图约束)
- 规范作为团队共享的"锚"
- TDD 作为底线 (Executable Spec) ——规范 (包括作为可执行规范的 TDD) 是在告诉代理要做什么, 明确它的边界



SDD 还没解决

- 多个代理怎么分工协作(编排)
- 代理出错了怎么自动纠正(反馈回路)
- 代理需要什么工具和权限(环境基础设施)

规范告诉代理**要做什么**,但没有告诉代理**怎么高效地做**。给工人一张蓝图但没有给他工厂——他知道要造什么,但缺乏工具、流水线和质检体系。

工具生态的割裂只是表象,根因是底层模型还在快速进化——今天为 **GPT-4 级代理设计的规范框架**,面对 **GPT-5 级代理可能需要重写**。

跃迁 4 — Karpathy 亲手"杀死"了 Vibe Coding，走向 80% 代理驱动

从 Vibe Coding 到 Agentic Engineering——软件工程的 9 级地震

"我现在 80% 的时间不再直接写代码，而是在编排代理。这标志着软件工程发生了一场 9 级大地震。"

Karpathy 的判断："LLM 代理能力在 2025 年 12 月跨过了连贯性阈值(Threshold of Coherence),引发了软件工程的相变。"



Karpathy 编排代理的时间

不再直接写代码

人类核心时间

架构设计、系统集成、业务断言

代理自主操作次数已从半年前的 10 次翻倍至 20 次，创业公司采纳率（33%）是企业（13%）的 2.5 倍。

相变已经发生，但如何应对？不同学派给出了截然不同的答案。

跃迁 4 — 发散的前沿：多学派并存

第四次跃迁尚未定局



前三次跃迁已经"盖棺定论"。第四次仍在"百家争鸣"——四个学派各执己见,正是因为下一代模型可能让某一派立刻过时。而且这四派远非全部,未来还会有新的学派涌现。

你的团队该站哪一派? 甚至该不该站队? 取决于你的项目本质、团队认知水平、和你能承受多大的变化速度。**没有标准答案, 只有持续的选择。**

跃迁 4 — 务实阵营：纪律派 × 治具派

代码是草稿，环境是治具——两条路殊途同归

纪律派（Osmani/Google）和治具派（OpenAI）看似路径不同，但共享同一个底层信念：人类必须保持对系统的控制力。纪律派从流程约束入手，治具派从环境约束入手——最终都指向“约束解空间”。

纪律派 — Osmani (Google)

- 从指挥者（Conductor）到编排者（Orchestrator）的演进路径
- 四条铁律：规范先于代码 | 小块迭代 | 代码即草稿 | 测试作为停止条件
- 核心信念：模型在变强，但人类的判断力不能退化

治具派 — OpenAI 实验

- 3人 + GPT-5 → 5个月 → 100万行代码（0人类手写）
- 四大治具：上下文工程 | 架构约束 Linter | AGENTS.md | GC Agents
- 核心信念：不靠无限自由度，靠约束解空间

务实阵营的共识：不管模型多强，约束和审查永远不能缺席——区别只在于约束的层次和粒度。

跃迁 4 — 激进阵营：激进派 × 大一统派

不要和大模型对赌——拥抱变化本身

激进派（Boris Cherny/Anthropic）和大一统派（Jeff Dean/Google DeepMind）都押注同一个方向：模型能力的增长速度会碾压一切人为框架。与其精心设计约束，不如让系统本身具备适应能力。

激进派 — Boris Cherny (Anthropic)

- 个人 100% AI 生成，IDE 已卸载；团队人均产出提升 150%
- 代理拓扑：给 Claude 一个 Spec + 看板，agent swarm 自建 ticket、自行认领
- 三条判断：脚手架就是技术债 | 指令要极简 | Agent 学会自建工具

大一统派 — Jeff Dean (Google DeepMind)

- 人均调度 50 个并行代理，本质上是微型外包 CEO
- 知识封装为“软件包”，手写代码专家的护城河正在消失
- 核心瓶颈从代码层转移到算力与能源调度层

激进阵营的共识：**不在代码层精雕细琢，在模型能力和算力杠杆上建立优势——今天的最佳实践，就是明天的技术债。**

跃迁 4 — 实战的收束：SDD + 编排的务实传承

多数真实商业团队的最佳落地点

将"规范驱动开发 (SDD)"与"多智能体编排 (Orchestration)"结合——不是折中，而是从前三次跃迁中提炼出的务实路径。

意图控制：SDD 锁定边界

没有 Spec 的编排，就是让多个实习生在没有蓝图的工地上同时开工。
SDD + TDD 锚定意图，让规范成为不可篡改的契约。

算力释放：多代理流水线消化 Spec

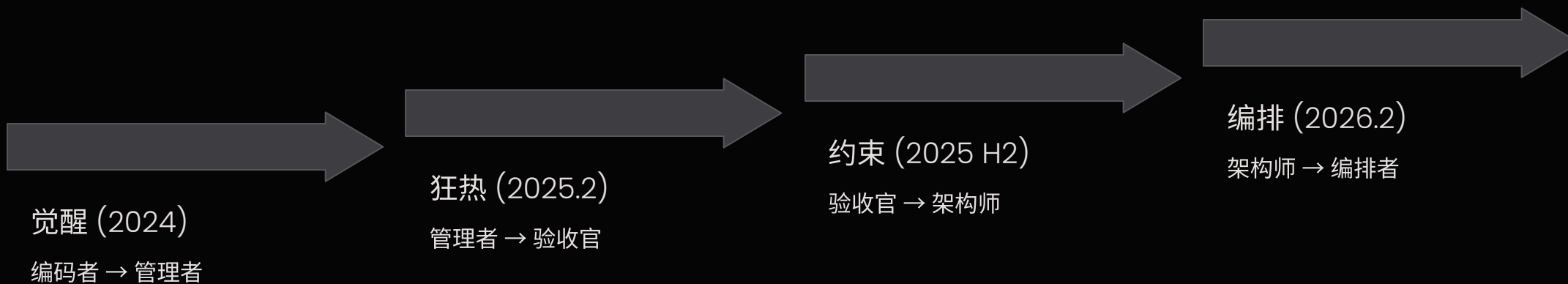
分析、编码、审查、部署——各由专职 Agent 流水线消化 Spec。人类彻底转向"写规范 + 调度流水线"。

❏ SDD 约束意图层 (what)，编排约束执行层 (how)。模型再怎么进化，这两个需求不会消失。

务实派的核心信念：**写好 Spec，编排好 Agent——意图和执行，缺一不可。**

四次跃迁之后，我们站在哪里？

不只是程序员的身份危机——是整个软件组织的范式重构



以上是个人层面的身份演化。但故事不止于此——当这些变化叠加到组织层面，冲击远比个人转型更剧烈。

四次跃迁重塑的不仅是“程序员写不写代码”这个问题。当3个人能产出过去30人团队的成果，当代码的保质期缩短到几个月——组织的底层逻辑正在被迫重写。接下来，我们从三个维度拆解这场组织级的范式重构：**经济模型的颠覆、安全边界的重划、以及控制逻辑的选择。**

代理时代的新现实（一）：经济模型的颠覆

10倍差距正在重写组织的成本结构

维度	过去	代理时代
核心成本	高薪开发者(线性扩编)	API Token + AgentOps 工具(算力杠杆)
人均产出	1-2 个 PR/天	10-20 个 PR/天
组织形态	庞大单一职能团队	极少数架构精英 + 多面手通才

先行者与后来者的差距已从百分比扩大到数量级。对CTO而言，这意味着团队规模不再是竞争力的代理变量——架构能力和代理编排能力才是。对非技术创始人而言，"招更多工程师"不再是默认答案。

代理时代的新现实（二）：安全边界与治理真空

代理越自主，风险越不可见

代理工程带来了生产力的跃升，但也把攻击面和责任边界炸开了。当代理能自主调用API、修改数据库、部署代码，传统的安全模型和治理框架都不够用了。



攻击面指数级扩大

自主代理拥有工具调用权限——一旦被提示注入攻击或误导，后果从"代码bug"升级为"系统级入侵"。传统代码审查无法覆盖代理的运行时决策。



责任归属的灰色地带

AI写了95%的代码，出了合规问题谁负责？开发者、AI供应商、还是部署决策者？法律框架和组织流程都还没跟上代理的自主程度。



最小权限原则的回归

三大学派在此达成罕见共识：代理必须在受限沙箱中运行，权限按任务粒度动态授予。"给代理最大自由度"的早期理念正在被纠正。

安全和治理不是技术团队的"附加题"——它们正在成为组织采纳代理工程的第一道门槛。没有安全框架的代理部署，就是在裸奔。

代理时代的新现实（三）：谁来掌舵？

不同组织需要不同的控制逻辑

四次跃迁告诉我们没有统一答案。但“没有标准答案”不等于“不需要选择”——每个组织都必须根据自身的风险承受力、团队认知水平和业务本质，建立自己的控制逻辑。



创业公司 / 新产品

高风险承受力，小团队，快速迭代。适合**激进路线**：让代理高度自主，代码视为临时产物，用速度换窗口期。

关键风险：技术债积累到无法重构的临界点。



成熟企业 / 关键系统

低风险承受力，大团队，合规要求高。适合**纪律派+治具派路线**：规范先行，沙箱运行，严格测试。

关键风险：采纳速度太慢，被先行者拉开数量级差距。



转型中的组织

大多数团队在这里。既不能全面激进，也不能原地不动。**务实路线**：从项目指令文件（L1 SDD）起步，逐步建立代理基础设施，按业务模块分级授权。

关键风险：内部认知分裂——一半人想冲，一半人想守。

控制逻辑不是一次性决策，而是持续校准。模型每隔数月跃进一次，你的控制框架也必须跟着进化——这本身就是代理工程的一部分。

结语：唯一确定的事

Karpathy 说："代理工程里面有艺术、科学和专业性。"

Boris Cherny 说："最重要的能力是新手心态——承认自己错了,然后重新出发。"

所有答案都有保质期

今天我们讨论的所有"答案"——无论是 SDD、治具工程，还是代理拓扑——都只是当下之解。

唯一不过期的能力

你的团队真正需要的，并非固定的正确答案，而是在持续变化中磨砺并建立自身判断力的能力。

在这个技术迭代如飞、代码保质期仅为数月的时代，唯一不会过期的，是你拥抱新知的速度，以及在不确定性中做出果断判断的勇气。